

Coding.Space

Autonome Code-Produktions-Pipeline und Vibe-Coding-Plattform für alle Sprachen — von Web-Apps bis C++ und Simulationen.

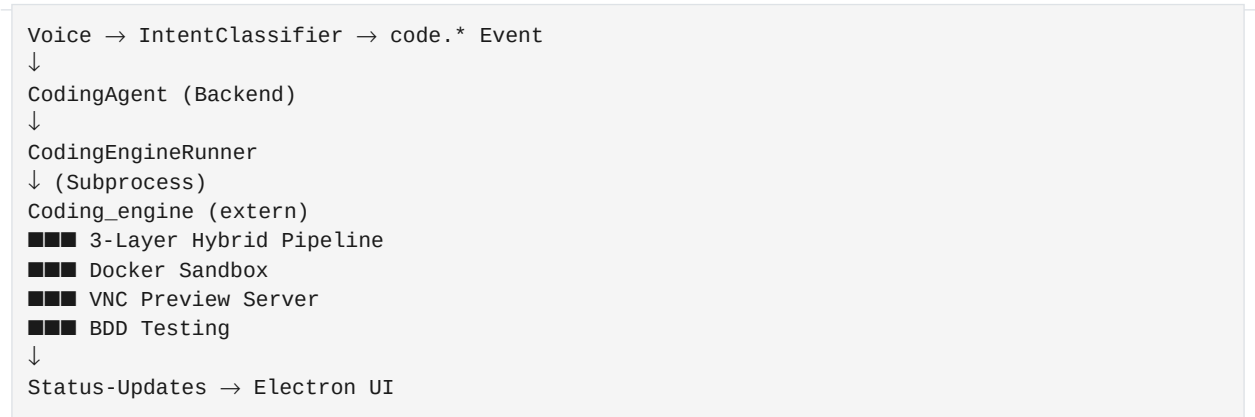
Overview

Coding.Space ist Vibeminds Software-Entwicklungsplattform mit zwei Modi:

1. **Autonome Pipeline:** Basierend auf einem Requirements-Stack generiert die Coding_engine (Society of Mind mit 40+ Agents) vollständige Projekte autonom — inklusive Docker-Sandbox, Testing und VNC-Preview.
2. **Vibe-Coding:** Ähnlich wie Lovable, aber sprachunabhängig — User beschreiben per Voice was sie wollen und sehen den Code in Echtzeit entstehen. Unterstützt nicht nur Web-Technologien, sondern auch C++, Simulationen und andere Software-Sprachen.

Die Architektur trennt klar: VibeMind steuert und überwacht, der externe Coding_engine generiert den Code.

Architektur



Backend-Agent

Datei: `python/spaces/coding/agents/coding_agent.py`

Event	Tool-Funktion	Beschreibung
<code>code.generate</code>	<code>generate_code</code>	Code-Generierung starten
<code>code.status</code>	<code>get_generation_status</code>	Generierungs-Status abfragen
<code>code.cancel</code>	<code>cancel_generation</code>	Generierung abbrechen
<code>code.list</code>	<code>list_generated_projects</code>	Projekte auflisten
<code>code.preview.start</code>	<code>start_preview</code>	VNC-Preview starten
<code>code.preview.stop</code>	<code>stop_preview</code>	Preview beenden

Event	Tool-Funktion	Beschreibung
<code>idea.to_project</code>	<code>idea_to_project_sync</code>	Bubble-Idee zu Projekt konvertieren
<code>code.modify</code>	<code>modify_code_sync</code>	Code per Voice modifizieren

Key Components

Komponente	Datei	Zweck
Backend-Agent	<code>agents/coding_agent.py</code>	Event-Routing zu Tools
Engine Runner	<code>engine/coding_engine_runner.py</code> (690 Zeilen)	Subprocess-Bridge zum Coding_engine
Project Discovery	<code>engine/project_discovery.py</code>	Projekt-Erkennung
Coding Tools	<code>tools/coding_tools.py</code> (587 Zeilen)	Haupt-Tool-Implementierungen
Adapted Tools	<code>tools/adapted_coding_tools.py</code>	Typisierte Wrapper
Voice Tools	<code>tools/voice_coding_tools.py</code>	Async Voice-Wrapper
Broadcast Agent	<code>broadcast/coding_broadcast_agent.py</code>	Electron-UI-Updates
Coding_engine	<code>Coding_engine/</code> (Git-Submodul)	Externer Code-Generator

CodingEngineRunner

Der Runner (`engine/coding_engine_runner.py`) verwaltet den externen Coding_engine als Subprocess:

- **VNC-Port-Allokation:** Unique Ports pro Job ab `BASE_VNC_PORT` (6080)
- **Requirements-Generierung:** Erstellt `requirements.json` aus Voice-Input
- **Echtzeit-Status-Streaming:** Parsed JSON + Text Progress-Updates
- **Quality-Monitoring:** Pollt `self_critique_report.json` während Generierung
- **4 Convergence-Modes:** `autonomous`, `strict`, `relaxed`, `fast`
- **Job-Lifecycle:** pending → generating → converging → testing → completed/failed

Subprocess-Flags:

```
python run_engine.py <requirements.json> \  
--autonomous --continuous-sandbox --external-sandbox \  
--enable-vnc --vnc-port 6080 --enable-validation \  
--output-dir <path> --json-progress --parallel 10
```

VNC-Preview-System

Voll implementiert mit Port-Management:

- `start_preview()` — Allokiert Port, startet VNC-Server, speichert URL in DB
- `stop_preview()` — Terminiert Server, gibt Port frei

- `_find_available_port()` — Verhindert Port-Kollisionen
- DB-Felder: `vnc_port` `INTEGER`, `preview_url` `TEXT` in `projects` Tabelle

Testing

Der externe `Coding_engine` enthält BDD-Testing-Infrastruktur:

- 23 Gherkin Feature-Files (User Stories)
- Pytest Step-Definitions mit Factories und Fixtures
- `seed_data.sql` für Test-Datenbanken
- `--enable-validation` Flag für automatische Tests

***Hinweis:** Test-Ergebnisse werden vom externen Subprocess verarbeitet. Voice-gesteuerte Test-Ausführung und Ergebnis-Anzeige in Electron sind noch nicht implementiert.*

Current Status

Implementiert

- Voice-gesteuerte Code-Generierung über Intent-Classification
- 8 Event-Types mit vollständigem Tool-Mapping
- `CodingEngineRunner` als Subprocess-Bridge
- VNC-Preview-System mit Multi-Project Port-Management
- Echtzeit-Status-Streaming zu Electron UI
- Job-Lifecycle-Management (Start, Cancel, Status)
- Quality-Monitoring via `self_critique_report.json`
- Docker-Sandbox-Integration (Flags an Engine)
- Projekt-Tracking in SQLite (`projects` Tabelle)
- Idea-to-Project Konvertierung aus Bubbles

Nicht implementiert

- Test-Ergebnisse nicht in Electron UI angezeigt
- Keine Voice-gesteuerte Einzeltest-Ausführung
- Kein Code-Review-Gate in VibeMind (extern im `Coding_engine`)
- Production-Deployment-Pipeline über Preview hinaus
- UI/UX-Design-to-Code-Konvertierung

Roadmap

- Test-Ergebnisse und Quality-Reports in Electron UI anzeigen
- Voice-gesteuerte Test-Ausführung und Debugging
- Production-Deployment-Infrastructure über Preview hinaus

- Spezialisierte Code-Generierung für Domain-spezifische Probleme
- Code-Review-System mit automatisierten Quality-Checks
- Progressive Refinement (iterative Verbesserung basierend auf Feedback)
- Containerized Environment Generation für komplexe Deployments

Ecosystem-Fit

Coding.Space transformiert Intent in Production Code. Ideas.Space artikuliert was gebaut werden soll, und Coding.Space generiert es. Rowboat.Space liefert Business-Logik und Daten-Requirements. Software Design.Space validiert Architektur-Entscheidungen. Desktop.Space validiert, dass deployed Applications korrekt funktionieren. The Brain.Space informiert architektonische Entscheidungen durch Pattern-Erkennung.